

中山大学专业课《程序设计I》

第九节 字符串 (上)

C Programming

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

目录

CONTENTS

01

字符与字符集

02

字符串概述

03

字符数组

04

字符串输入输出

05

字符串基本操作

字符 (char)

```
#include <stdio.h>

int main() {
    char c0 = 'A';        // 字符由单引号' '包围
    char c1 = 65;         // 字符是一个数字
    char c2 = '\n';       // 特殊字符可用转义
    char c3 = '\x41';      // 用16进制表示字符
    char c4 = '\101';      // 用8进制表示字符
    printf("%c,%c%c", c0, c1, c2);
    printf("%c,%c", c3, c4);
}
```

输出

```
A,A
A,A
```

字符是特殊的整数类型

ASCII表：字符与数字的映射

- 美国信息交换标准代码
- American Standard Code for Information Interchange / ASCII
- 共 128 个编码
- 历史
 - 用于电传打印机 (teletype)
 - 也称远程打印
- 原理
 - 按键 A, 发送 65
 - 收到 65, 打印 A
 - 按键 ^G, 发送 7
 - 收到 7, 响铃提示
 - 按键 ^M, 发送 13
 - 收到 13, 打印头回零位
 - 按键 ^J, 发送 10
 - 收到 10, 打印头下移一行
 - 按键 ^D, 发送 4
 - 收到 4, 电报传输结束

ASCII 字符代码表 一																						
高四位 低四位		ASCII非打印控制字符										ASCII 打印字符										
		0000					0001					0010	0011	0100	0101	0110	0111					
		0					1					2	3	4	5	6	7					
	+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	ctrl	
0000	0	0	BLANK NULL	^@	NUL 空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p
0001	1	1	☺	^A	SOH 头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q
0010	2	2	☺	^B	STX 正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r
0011	3	3	♥	^C	ETX 正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s
0100	4	4	◆	^D	EOT 传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t
0101	5	5	♣	^E	ENQ 查询	21	§	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u
0110	6	6	♠	^F	ACK 确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v
0111	7	7	●	^G	BEL 震铃	23	↑↓	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w
1000	8	8	◼	^H	BS 退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x
1001	9	9	○	^I	TAB 水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y
1010	A	10	◻	^J	LF 换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z
1011	B	11	♂	^K	VT 竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{
1100	C	12	♀	^L	FF 换页/新页	28	└	^\ FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124		
1101	D	13	🎵	^M	CR 回车	29	↔	^] GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}	
1110	E	14	🎵	^N	SO 移出	30	▲	^_	RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~
1111	F	15	☼	^O	SI 移入	31	▼	^-	US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ Back space

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键” 输入

ASCII表基本特征

- **控制符（共33个）**

- 前32个字符，例如：9->'\\t'; 10->'\\n'; 13->'\\r'
- 127是控制符，DEL

- **可打印字符（共95个）**

- 32是空格
- 48-57是'0'... '9'
- 65-90是'A'... 'Z'
- 97-122是'a'... 'z'

- **空白字符（共5个）**

- 空格、水平制表、垂直制表、换页、换行

小知识：

不同操作系统下换行的表示：

\\n: UNIX 系统行末结束符

\\r\\n: window 系统行末结束符

\\n: macOS 基于UNIX

我们经常遇到的一个问题就是，Unix/Mac系统下的文件在Windows里打开的话，所有文字会变成一行；而Windows里的文件在Unix/Mac下打开的话，在每行的结尾可能会多出一个^M符号。

ASCII 的扩展字符集

- **扩展ASCII**

- 对 128-255 定义了西方常用字母
- 例如: 'ä' 的编码 '\xe4' (十进制 228)

- **大字符集, 16位编码**

- JIS 日文字母与日文汉字
- Big5 港台繁体汉字
- GB2312/GBK 国标
- ANSI windows 大字符集标准
 - 中国 CP936, 即 GBK
 - 日本 CP932, 即 JIS

小知识：

大字符集一般用两个扩展**ASCII**表示一个字母（汉字）。这样，ASCII 和 汉字 混在一起也容易识别。

为了与文字传输、控制设备兼容，仅可打印字符区域对应区域的扩展**ASCII**可用于大字符集的编码。

因历史原因，各地编码并不统一。使用不当编码打开文档，会产生乱码。

GBK编码举例

- ❑ 假设内存中有以下字节序列（用十进制表示）：

72, 105, 211, 248, 207, 223, 0

- ❑ 程序解析过程如下：

1. 72: < 128, 解析为 ASCII 字符 'H'
2. 105: < 128, 解析为 ASCII 字符 'i'
3. 211: > 128, 程序知道这是一个汉字的开始。
4. 248: 程序立即读取下一个字节, 将 (211, 248) 组合起来, 查 GBK 码表 -> “你”
5. 207: > 128, 程序知道这是另一个汉字的开始
6. 223: 程序立即读取下一个字节, 将 (207, 223) 组合起来, 查 GBK 码表 -> “好”
7. 0: < 128, 解析为 ASCII 字符 \0 (字符串结束符)

- ❑ 最终, 程序显示为: **Hi你好**

Unicode（单一编码）字符集

- Unicode是一个字符集，它是所有其他广泛使用字符集的超集，包含了来自ISO/IEC 6937、ISO/IEC 8859家族、**Big5**、KS X 1001、JIS X 0213、**GB 2312**、**GBK**、GB 18030、HKSCS和CNS 11643等字符集的字符
- 它包含三种编码形式
 - UTF-8，变长格式
 - UTF-16，16位宽字符
 - UTF-32，32位宽字符

UTF-16 部分字符编码

Code (Hex)	Character	Source
0041	A	English (Latin)
042F	Я	Russian (Cyrillic)
OE09	฿	Thai
13EA	Ꮝ	Cherokee
211E	℞	Letterlike Symbols
21CC	⇒	Arrows
282F	⠆	Braille
345F	𐀀	Chinese/Japanese/ Korean (Common)

C语言字符类型与常量

- 单字节变量，表示英文或西方字符
 - char, 一个字节
 - 例如: char c='a'; 或 char c='\13';
- 16 位宽字符，表示中文，日文等
 - 类型 char16_t, 两个字节无符号整数
 - 例如 u'字' 但非 u'👉' (u'\U0001f34c')
- 32 位宽字符，表示 emoji 等扩展
 - 类型 char32_t, 四个字节无符号整数
 - 例如 U'字' 或 U'👉'
- 宽字符
 - 类型 wchar_t, 设定为16位或32位
 - 例如 L'β' 或 L'字'

转义符	含义
\\	\ 字符
\'	' 字符
\"	" 字符
\?	? 字符
\a	警报铃声
\b	退格键
\f	换页符
\n	换行符
\r	回车
\t	水平制表符
\v	垂直制表符
\ooo	一到三位的八进制数
\xhh ...	一个或多个数字的十六进制数
\uhhhh...	UTF-16或UTF-32编码

代码范例参见：[字符常量 - cppreference.com](http://cppreference.com)

举例：常见字符集编码的区别

字符	ASCII	UTF-16	UTF - 8	GBK
A	41	00 41	41	-
汉	-	6C 49	E6 B1 89	BA BA

当字符串中包含汉字时，如果编译程序默认时**UTF-8**编码，每个汉字占**3个字节**；如果默认是**ANSI**，即GBK，则每个汉字**2个字节**

如果你在2000年前出生，那么出生证上无法打出“燊”字，因为DOS使用的是GB2312，只有6000个常用汉字；Windows 95 开始，GBK 包括了GB2312，同时又增加了近20000个新的汉字（包括繁体字）和符号

字符串 (String)

C语言**字符串**是尾随**零值字节**（空字符'\0'）的**非零字节序列**

例如: "aabbccdd", "wkfueh@ei~", "t ", "Hello World."

```
#include <stdio.h>
#include <string.h>
int main(void) {
    // 字符串常量由双引号" "包围
    // "Hello"由6个字符构成: 'H' 'e' 'l' 'l' 'o' '\0'
    // "孙悟空"由10个, 或7个字符构成?
    // 字符串常量中可用\ooo代表字符, o为八进制数
    printf("%s %s %s!\n", "Hello", "孙悟空", "孙\0悟空"); // 使用%s格式化
}
```

printf遇到第一个\0就结束输出

字符串 (String)

字符是组成字符串的基本单元

例如"Hello"包含的字符依次是'H','e',' ','l','l','o','\0'

字符串其实是一种特殊的字符数组，其必须以'\0'作为结束标记

字符串的储存与初始化

C语言使用**字符数组**存储字符串，字符串常量可以初始化字符数组

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char your_name[4] = "Bob";
    printf("Hello, %s!\n", your_name);
}
```

字符串常量最后以'\0'作为结束字符，需保证数组长度**始终大于**字符串实际长度

字符串的储存与初始化

字符串如果比较长，可采用**续行符**或字符串**拼接**方式拼接字面量

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char str[100] = "hello \
\n\
" "world!\n";
}
```

自然行最后 **** 是续行字符

使用字符数组存储字符串，声明**100**最多保存**长度为99**的字符串，最后 **\0** 占一个字节

编译预处理期自动拼接用空白符间隔的字面量

str[50]是什么? '\0'

字符串的储存与初始化

`char ch = 'H'; // 正确`

`char ch = "H"; // 错误`

`// 可以用多个字符或一个字符串对char类型的数组元素进行初始化`

`char msg[3] = {'H', 'i', '!'}; // 不是字符串`

`char msg[4] = {'H', 'i', '!', '\0'};`

`char msg[4] = "Hi!";`

`char msg[ ] = "Hi!"; // msg有多大?`

H	i	!	\0
---	---	---	----

`// 最后三种声明是等价的字符串`

对不对?



```
char msg[4];  
msg = "Hi!";
```



```
char msg[4];  
msg[0] = 'H';  
msg[1] = 'i';  
msg[2] = '!';  
msg[3] = '\0';
```

字符数组与字符串的区别

考虑以下两种char类型的数组：

1) `char msg[3] = {'H', 'i', '!'};`

2) `char msg[4] = {'H', 'i', '!', '\0'};`

第一种没有结束符，是一般的char类型数组，不被认为是字符串

第二种有结束符，被认为是字符串

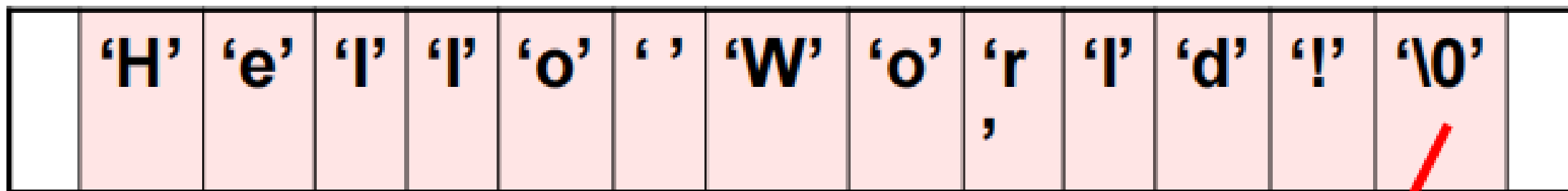
C标准库中有很多函数可以整体处理这种数组（及字符串常量）

内存中的字符串

双引号" "括起来的字符序列称为字符串常量/字面量，例如：

"Hello World!", "I am a teacher\n"

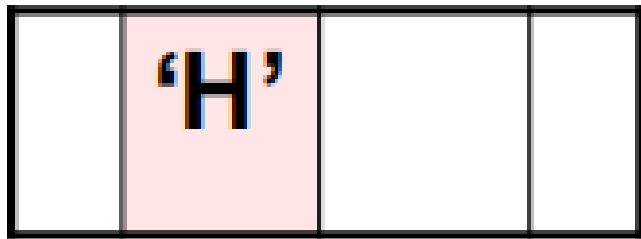
字符串常量在内存中的存储：



结束符

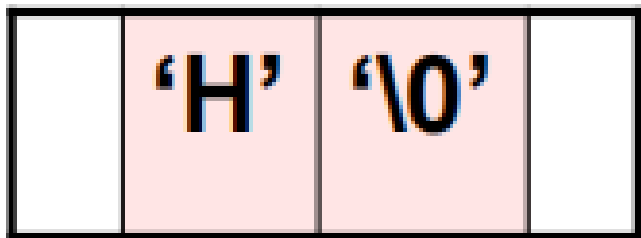
内存中的字符与字符串

字符'H'在内存中的存储为：



占一个字节

而字符串"H"在内存中的存储为：



占两个字节（数组）

字符串是一种特殊的字符数组

字符串的性质

□ **字符串长度**：一个字符串所**包含有效字符**的数目

■ 例如："efl35"的长度为**5**，"wofskl6"的长度为**7**，"12345"的长度为**5**

□ **空串**：长度为0的字符串，""，记为 $\emptyset$

□ **子串**：由某个字符串中**连续的一段字符**组成的字符串

■ 例如"abcd"的所有子串为 $\emptyset$ ，"a"，"b"，"c"，"d"，"ab"，"bc"，"cd"，"abc"，"bcd"，**"abcd"**

字符串的性质

□ **前缀**：某个字符串**开头**的一段子串

■ 例如 “abcd” 的所有前缀为 $\emptyset$, "a", "ab", "abc", "abcd"

■ 长度为 n 的字符串共有 $n + 1$ 个前缀

□ **公共（最长）前缀**：几个字符串共有的前缀

■ 例如： "abcd", "about", "abuse"的公共前缀是"ab"

□ **后缀**：某个字符串末尾的一段子串

■ 例如 “abcd” 的所有后缀为 $\emptyset$, "d", "cd", "bcd", "abcd"

■ 长度为 n 的字符串共有 $n + 1$ 个后缀

字符串输入输出：格式化输出

```
char str1[] = "This is a string.";
char str2[] = {'H', 'e', 'l', 'l', 'o'};
printf("%s\n\n", str1); // 字符串整体输出用%s
printf("%s\n\n", str2);
```

This is a string.

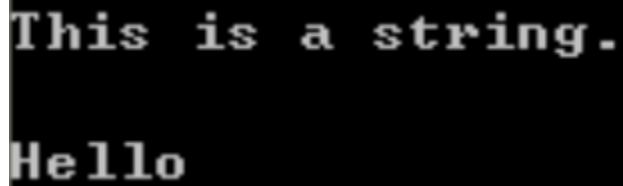
Hello烫讓his is a string.

str2没有结束符，输出将一直进行直到遇到某个字节的值刚好为结束符为止（具有随机性）！

因此对于str2，正确的输出方法应该是利用**循环逐个输出元素**

字符串输入输出：格式化输出

```
char str1[] = "This is a string.";
char str2[] = {'H', 'e', 'l', 'l', 'o', '\0'};
printf("%s\n\n", str1); // 字符串整体输出用%s
printf("%s\n\n", str2);
```

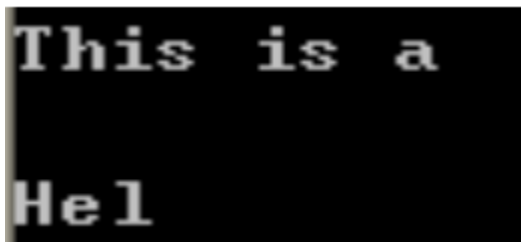


```
This is a string.
Hello
```

str1和str2都有结束符，输出至结束符处结束

字符串输入输出：格式化输出

```
char str1[] = "This is a string.";
char str2[] = {'H', 'e', 'l', '\0', 'l', 'o', '\0'};
printf("%s\n\n", str1); // 字符串整体输出用%s
printf("%s\n\n", str2);
```



```
This is a
Hel
```

字符串的输出遇到**第一个结束符**时就结束

字符串输入输出：格式化输出

- ❑ 字符串可以利用%s整体输出
- ❑ 字符数组 (int\float\double等) 不能整体输出，只能利用循环逐元素输出

字符串输入输出：格式化输入

```
char str[10];  
  
printf("Please enter a string less than 9 characters:\n");  
  
scanf("%s", str); // 注意不需要取地址运算符&  
  
printf("The string you enter is: %s\n\n", str);
```

字符串可以利用%s整体输入，但对于其它类型的数组，例如int\float\doulbe等，不能整体输入，只能利用循环逐个元素输入

字符串输入输出：格式化输入

```
char str[10];  
printf("Please enter a string less than 9 characters:\n");  
scanf("%s", str); // 注意不需要取地址运算符&  
printf("The string you enter is: %s\n\n", str);
```

```
Please enter a string less than 9 characters:  
student  
The string you enter is: student
```

键盘输入到 't' 时 Enter，将结束输入，并在't'后置入空字符

str在内存中（? 代表值不确定）：

	's'	't'	'u'	'd'	'e'	'n'	't'	'\0'	?	?	
--	-----	-----	-----	-----	-----	-----	-----	------	---	---	--

字符串输入输出：格式化输入

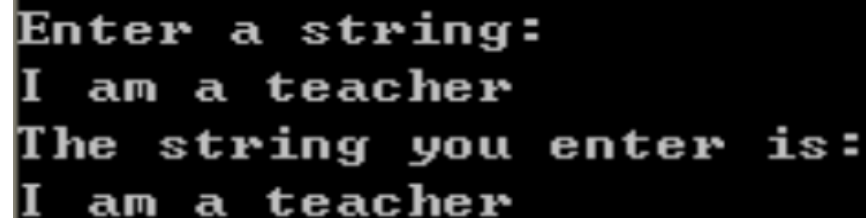
```
char str[10];  
  
printf("Please enter a string less than 9 characters:\n");  
scanf("%s", str); // 注意不需要取地址运算符&  
printf("The string you enter is: %s\n\n", str);
```

```
Please enter a string less than 9 characters:  
stu dent  
The string you enter is: stu
```

在遇到**空白字符**时，仅获得空白字符之前的字符串

字符串输入输出：行操作 (f)gets/(f)puts

```
char str[100];  
  
puts("Enter a string:\n"); // 输出字符串  
  
gets(str); // 输入字符串  
  
puts("The string you enter is:\n");  
  
puts(str);
```



```
Enter a string:  
I am a teacher  
The string you enter is:  
I am a teacher
```

- (1) **gets**获得一整行字符串（包括空格，不包含'\n'），并写入'\0'
- (2) **puts**可以更快输出整行并换行，但失去了格式化的便利，如对齐等
- (3) 更安全的选择：**fgets()**

字符串练习1：选菜单

```
#include<stdio.h>
#include<string.h>

void print_menu(void){
    printf("\n1. 上楼 \n2. 下楼 \n3. 遛弯 \n4. 静思 \nn. 退出");
    printf("\n\n请输入 1..4 选择菜单, 输入 no 退出程序:");
}

int main(void) {
    char s[80];
    while(1){
        print_menu();
        gets(s);
        if (s[0]=='\n') return 0;
        if (s[0]>='1'&& s[0]<='4')
            printf("执行菜单 %d\n", s[0]-'0');
    }
}
```

字符串练习2：逆转字符串

逆转字符串，就是将一个字符串头尾倒置，变成一个新的字符串。

例如：abcde -> edcba

```
#include<stdio.h>
#include<string.h>
```

包含 string.h

```
int main(void) {
    char str[100];
    int len,t;
    gets(str);
```

确保字符数组足够大

整行且能包含空白字符的字符串使用

```
    for(len=0;str[len]!=0;len++);
    for(int i=0,j=len-1;i<j;i++,j--)
        t=str[i],str[i]=str[j],str[j]=t;
```

计算字符串长度

双指针法实现头尾字符交换

```
    printf("%s,%d",str,len);
    return 0;
```

```
}
```

字符串操作：求字符串长度strlen()

```
#include <stdio.h>
#include <string.h>
int main(void) {
    char str[] = "Hello!";
    unsigned int l1,l2;

    l1= strlen(str);
    l2= strlen("C PROGRAMMING!");
    printf("l1 = %u, l2 = %u\n\n", l1, l2);
    //strlen返回字符串的长度
}
```

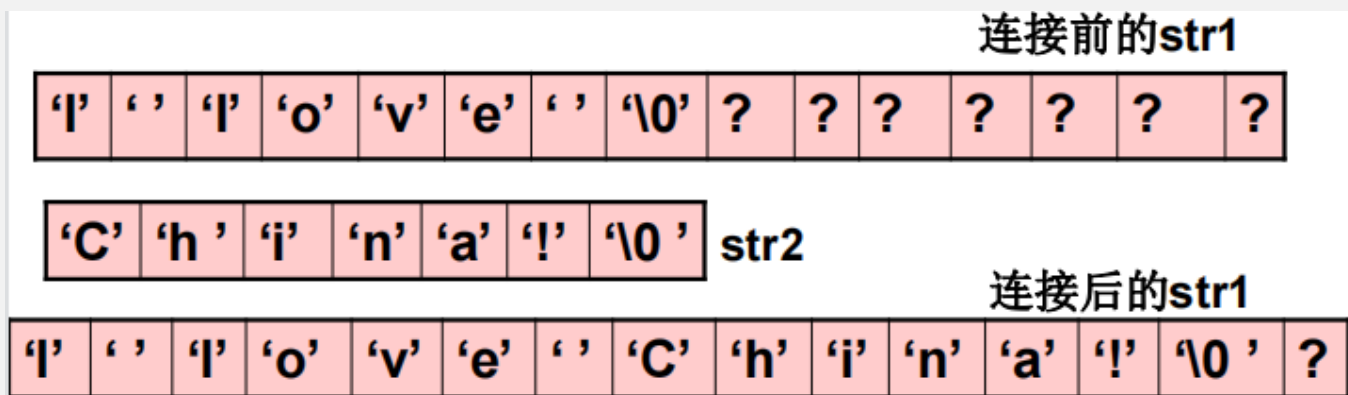
l1 = 6, l2 = 14

字符串操作：字符串连接strcat(toStr, fromStr)

- (1) 去掉str1的结束符，并将str2连接到str1上
- (2) **str1数组必须够长**，否则产生不可知错误（缓存区溢出）
- (3) 返回**str1数组的首地址**，即str1

```
#include <string.h>
#include <stdio.h>
```

```
int main(void) {
    char str1[15] = {"I love "};
    char str2[] = {"China!"};
    printf("%s", strcat(str1, str2));
}
```

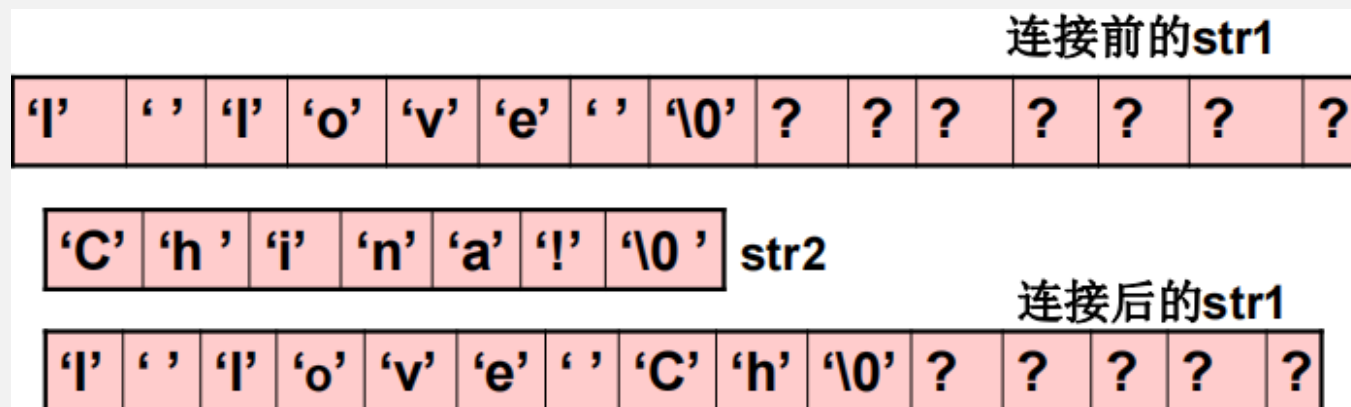


字符串操作：字符串连接strncat(toStr, fromStr)

把fromStr的前n个字符连接到toStr上，其余与strcat()相同

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[15] = {"I love "};
    char str2[] = {"China!"};
    printf("%s", strncat(str1, str2, 2));
}
```



字符串操作：字符串拷贝strcpy(toStr, fromStr)

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[15];
    char str2[] = "I love China!";
    strcpy(str1, str2);
    strcpy(str1, "You are a student.");
    // strcpy将fromStr包括空字符'\0'拷贝到toStr
    strncpy(str1, str2, 4);
    // strncpy将fromStr的头n个字符拷贝到toStr开始的位置, 注意：不添加空字符
}
```

字符串操作：字符串的比较strcmp(str1, str2)

```
#include<string.h>
#include<stdio.h>
int main(void) {
    char str1[] = "Hell";
    char str2[] = "Hello";
    printf("%d\n", strcmp(str1, str2));
    strcpy(str1, "ABcD");
    strcpy(str2, "ABCD");
    printf("%d\n", strcmp(str1, str2) );
    printf("%d\n", strcmp("fine", "fine"));
}
```



str1和str2按照**字符在字符集中的表示值**进行比较：从第一个字符开始，逐个字符两两进行比较；第一对不同的字符出现时，较大字符所在的字符串就是大者；如果其中较短的字符串**是较长字符串前面的子字符串**，则较长的字符串大于较短的字符串。

函数返回值：

str1 **小于** str2时，返回**小于0**的整数值

str1 **等于** str2时，返回整数值**0**

str1 **大于** str2时，返回**大于0**的整数值

字符串操作：字符串比较strncmp(str1, str2, n)

比较str1与str2头n个字符的大小，返回值与strcmp相同

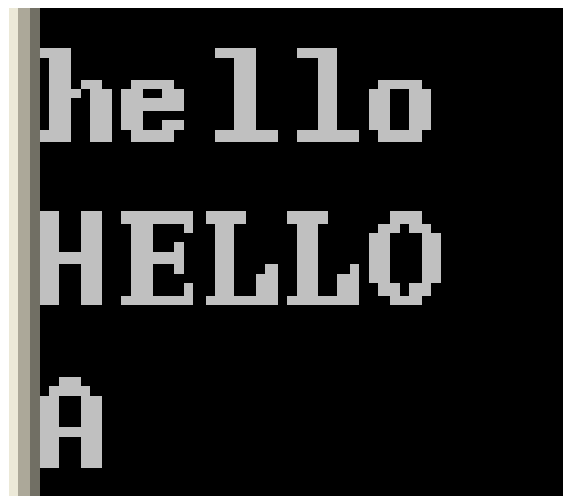
字符串操作——大小写转换

strlwr(str): 将字符串str的字母全部转化为**小写**

strupr(str): 将字符串str的字母全部转化为**大写**

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str1[] = "Hello";
    char ch = 'a';
    strlwr(str1); puts(str1);
    strupr(str1); puts(str1);
    strupr(&ch);
    printf("%c\n\n", ch);
}
```



hello
HELLO
A

字符串操作：字符检索strchr(str, c)

`char *strchr(const char *str, int c)` 在参数 `str` 所指向的字符串中搜索指向**第一次出现字符c**（一个无符号字符）位置的**指针**

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str[] = "abcdedcbaf";
    printf("%d\n", strchr(str, 'd') - str);
    // 'd'在str中第一次出现的下标
    printf("%s\n", strchr(str, 'd'));
    // 从第一次'd'出现开始的字符串
}
```



3
dedcbaf

字符串操作——字符串检索strstr(str1, str2)

char *strstr(const char *str1, const char *str2) 在字符串 str1 中查找指向**第一次出现字符串 str2 位置的指针(不包含终止符 '\0')**。

```
#include <string.h>
#include <stdio.h>

int main(void) {
    char str[] = "abcdedcbaf";
    printf("%d\n", strstr(str, "ded") - str);
    // "ded" 在 str 中第一次出现的下标
    printf("%s\n", strstr(str, "ded"));
    // 从第一次 "ded" 出现开始的字符串
}
```



3
dedcbaf

字符串练习3：打印千分位

例如：给定整数 n ，请打印该整数，千分位用 ',' 表示。

输入1: 1234 输出1: 1,234; 输入2: -123456 输出2: -123,456

```
int main() {
    int d; // 千分位分隔符
    char s[N_LEN] = {0};

    scanf("%d",&d);
    int q=d<0?-d:d;
    int p=0,sep=0;
    while(q!=0) {
        if (sep==3) {
            s[p++]=T_SEP;
            sep=0;
        }
        sep++;
        s[p++]=q%10+'0';
        // printf("%c\n",s[p-1]);
        q=q/10;
    }
    if (d<0) s[p++]='-';
    else if (d==0) s[p++]='0';
    s[p]=0;

    reverse(s);

    printf("%s",s);
}
```

请解释变量 q,p,sep 的含义

对负数和零特殊处理

总结

操作	代码	操作	代码
输入	<code>scanf("%s" , str);</code>	字符串的拷贝	<code>strcpy(toStr, fromStr)</code>
输出	<code>printf("%s" , str);</code>	字符串的比较	<code>strcmp(str1, str2)</code> <code>strncmp(str1, str2, n)</code>
整行输入	<code>gets(str);</code>	转换为小写	<code>strlwr(str)</code>
整行输出	<code>puts(str);</code>	转换为大写	<code>strupr(str)</code>
获取/操作第i个字符	<code>str[i]</code>	字符检索	<code>strchr(str, c)</code>
求字符串str长度	<code>strlen(str)</code>	字符串检索	<code>strstr(str1, str2)</code>
字符串的连接	<code>strcat(toStr, fromStr)</code> <code>strncat(toStr, fromStr, n)</code>	字符串结束标志	<code>'\0'</code> (ASCII码为0的字符)

谢谢！

邮箱: yuesh5@mail.sysu.edu.cn